L&T CIA-1 PRACTICAL

BY: AMOGH V HOSAMANI

REG NO:2462544

CLASS:5BTCSIOT

GITHUB LINK - https://github.com/amoghvh/L-t

Task 1:Hospital Patient System – Welcome Message

Question – Write a program that prints a welcome message for the "Hospital Patient System" to the console.

Answer –

```
// Task 1: Welcome Message
console.log("Welcome to the Hospital Patient System!");
console.log("=======================================");
```

```
[Done] exited with code=0 in 0.105 seconds

[Running] node "d:\SEM5\CW L&T lab\l&t.js"
Welcome to the Hospital Patient System!
=======================================
```

Brief Description:

This task involves creating a simple Node.js program that displays a welcome message for the Hospital Patient System. The program uses console.log() to print the welcome message, establishing the foundation for the hospital management system. This demonstrates basic output operations in JavaScript and serves as an introductory greeting for users accessing the system.

Task 2: Hospital Patient System — Declare and Update Variables

Question - Declare variables to store the system name and the total number of patients, then print both.

Answer –

```
JS l&t.js l&t.js\...
1   // Task 2: Declare and Update Variables
2   let systemName = "Christ Hospital Patient Management System";
3   let totalPatients = 150;
4
5   console.log("System Name:", systemName);
6   console.log("Total Patients:", totalPatients);
7
8   // Updating variables
9   systemName = "Christ University Hospital System";
10  totalPatients = 175;
11
12  console.log("\nAfter Update:");
13  console.log("System Name:", systemName);
14  console.log("Total Patients:", totalPatients);
```

Output:

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
System Name: Christ Hospital Patient Management System
Total Patients: 150

After Update:
System Name: Christ University Hospital System
Total Patients: 175

[Done] exited with code=0 in 0.115 seconds
```

Brief Description:

This task focuses on variable declaration and manipulation in JavaScript. Variables are declared using let to store the system name (string) and total number of patients (number). The values are printed initially and then updated to demonstrate variable reassignment. This showcases understanding of dynamic variables, data storage, and the ability to modify values during program execution.

Task 3: Hospital Patient System — Identify Datatypes

Question - Declare variables of type Number, String, and Boolean related to a patient, and print their datatypes using typeof.

Answer –

```
JS l&t.js l&t.js\...
1   // Task 3: Identify Datatypes
2   let patientName = "John Doe";        // String
3   let patientAge = 45;                 // Number
4   let isAdmitted = true;               // Boolean
5   let patientId = "P-2026-001";        // String
6   let bedNumber = 205;                 // Number
7   let hasInsurance = false;            // Boolean
8
9   console.log("Patient Name:", patientName, "| Type:", typeof patientName);
10  console.log("Patient Age:", patientAge, "| Type:", typeof patientAge);
11  console.log("Is Admitted:", isAdmitted, "| Type:", typeof isAdmitted);
12  console.log("Patient ID:", patientId, "| Type:", typeof patientId);
13  console.log("Bed Number:", bedNumber, "| Type:", typeof bedNumber);
14  console.log("Has Insurance:", hasInsurance, "| Type:", typeof hasInsurance);
```

Output:

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
Patient Name: John Doe | Type: string
Patient Age: 45 | Type: number
Is Admitted: true | Type: boolean
Patient ID: P-2026-001 | Type: string
Bed Number: 205 | Type: number
Has Insurance: false | Type: boolean

[Done] exited with code=0 in 0.177 seconds
```

Brief Description:

This task emphasizes understanding JavaScript data types and the typeof operator. Multiple variables representing patient information (name, age, admission status, etc.) are declared with different data types. The typeof operator is used to identify and print the data type of each variable, demonstrating the ability to work with primitive data types and type checking in JavaScript.

Task 4: Hospital Patient System — Perform a Calculation

→ Write a program to calculate the age difference between two patients using arithmetic operators, and print the result.

Answer –

JS l&t.js l&t.js\...

```javascript
// Task 4: Perform a Calculation
let patient1Age = 68;
let patient2Age = 45;

let ageDifference = patient1Age - patient2Age;
let absoluteAgeDifference = Math.abs(ageDifference);

console.log("Patient 1 Age:", patient1Age, "years");
console.log("Patient 2 Age:", patient2Age, "years");
console.log("Age Difference:", absoluteAgeDifference, "years");

// Additional arithmetic operations
let totalAge = patient1Age + patient2Age;
console.log("Total Age of both patients:", totalAge, "years");

let averageAge = totalAge / 2;
console.log("Average Age:", averageAge, "years");
```

Output –

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
Patient 1 Age: 68 years
Patient 2 Age: 45 years
Age Difference: 23 years
Total Age of both patients: 113 years
Average Age: 56.5 years

[Done] exited with code=0 in 0.123 seconds
```

Brief Description:

This task involves performing arithmetic operations in JavaScript. Two patient ages are declared as variables, and the age difference is calculated using subtraction. Additional calculations like total age and average age are also demonstrated using arithmetic operators. This showcases understanding of mathematical operations, variable manipulation, and result formatting in JavaScript.

Task 5: Hospital Patient System — Classify a Record

→ Write a program to check whether a given patient's `age` meets a specific condition using if-else.

Answer –

JS l&t.js l&t.js\...

```javascript
// Task 5: Classify a Record
let patientAge = 72;

console.log("Patient Age:", patientAge);

if (patientAge < 18) {
    console.log("Classification: Pediatric Patient");
} else if (patientAge >= 18 && patientAge < 60) {
    console.log("Classification: Adult Patient");
} else if (patientAge >= 60 && patientAge < 80) {
    console.log("Classification: Senior Citizen Patient");
} else if (patientAge >= 80) {
    console.log("Classification: Geriatric Patient");
} else {
    console.log("Classification: Invalid Age");
}

// Additional condition check
if (patientAge >= 65) {
    console.log("Eligible for Senior Citizen Benefits: Yes");
} else {
    console.log("Eligible for Senior Citizen Benefits: No");
}
```

Output –

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
Patient Age: 72
Classification: Senior Citizen Patient
Eligible for Senior Citizen Benefits: Yes

[Done] exited with code=0 in 0.111 seconds
```

Brief Description:

This task implements conditional logic using if-else statements to classify patients based on their age. Different age ranges (pediatric, adult, senior citizen, geriatric) are defined, and the program determines which category a patient falls into. Additional conditions like eligibility for senior citizen benefits are also checked, demonstrating the use of comparison operators and multi-level decision-making in JavaScript.

Task 6: Hospital Patient System — Iterate Over Records 2 Marks

→ Create an array of at least 4 patient objects and use a loop to print each one's details.

Answer –

JS I&t.js I&t.js\...

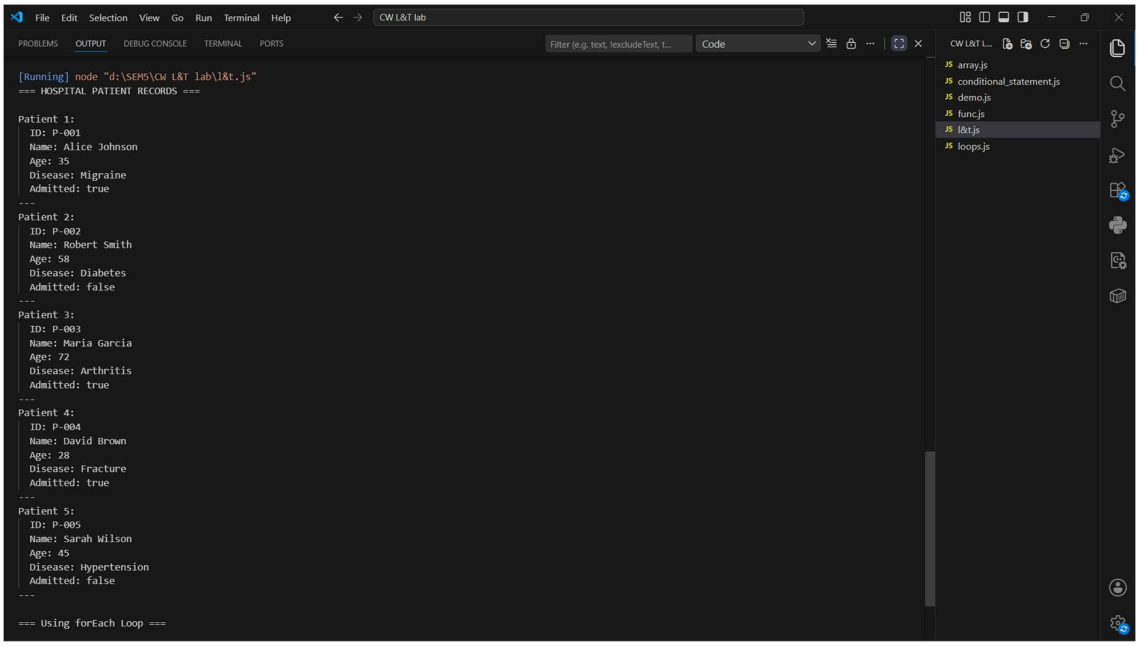
```
// Task 6: Iterate Over Records
const patients = [
    {
        id: "P-001",
        name: "Alice Johnson",
        age: 35,
        disease: "Migraine",
        admitted: true
    },
    {
        id: "P-002",
        name: "Robert Smith",
        age: 58,
        disease: "Diabetes",
        admitted: false
    },
    {
        id: "P-003",
        name: "Maria Garcia",
        age: 72,
        disease: "Arthritis",
        admitted: true
    },
    {
        id: "P-004",
        name: "David Brown",
        age: 28,
        disease: "Fracture",
        admitted: true
    },
    {
        id: "P-005",
        name: "Sarah Wilson",
        age: 45,
        disease: "Hypertension",
        admitted: false
    }
];

console.log("=== HOSPITAL PATIENT RECORDS ===\n");

for (let i = 0; i < patients.length; i++) {
    console.log(`Patient ${i + 1}:`);
    console.log("  ID:", patients[i].id);
    console.log("  Name:", patients[i].name);
    console.log("  Age:", patients[i].age);
    console.log("  Disease:", patients[i].disease);
    console.log("  Admitted:", patients[i].admitted);
    console.log("---");
}

// Using forEach loop
console.log("\n=== Using forEach Loop ===\n");
patients.forEach((patient, index) => {
    console.log(`${index + 1}. ${patient.name} (${patient.age}) - ${patient.disease}`);
});
```

Output –



Brief Description:

This task involves working with arrays and objects in JavaScript. An array containing multiple patient objects (each with properties like id, name, age, disease, and admission status) is created. Both for loop and forEach method are used to iterate through the array and print each patient's details. This demonstrates understanding of data structures, object creation, and array iteration techniques.

Task 7: Hospital Patient System — Skip or Stop Early 2 Marks

→ Using the array of patients, use break or continue while looping to skip or stop at a specific condition on `age`.

Answer –

JS l&t.js l&t.js\...

```
// Task 7: Skip or Stop Early

// Patient array from Task 6 (reused)
const patientArray = [
    { id: "P-001", name: "Alice Johnson", age: 35, disease: "Migraine", admitted: true },
    { id: "P-002", name: "Robert Smith", age: 58, disease: "Diabetes", admitted: false },
    { id: "P-003", name: "Maria Garcia", age: 72, disease: "Arthritis", admitted: true },
    { id: "P-004", name: "David Brown", age: 28, disease: "Fracture", admitted: true },
    { id: "P-005", name: "Sarah Wilson", age: 45, disease: "Hypertension", admitted: false }
];

console.log("=== USING CONTINUE (Skip admitted patients) ===\n");

for (let i = 0; i < patientArray.length; i++) {
    if (patientArray[i].admitted === true) {
        continue; // Skip admitted patients
    }
    console.log("Non-admitted Patient:", patientArray[i].name);
}

console.log("\n=== USING BREAK (Stop at age > 60) ===\n");

for (let i = 0; i < patientArray.length; i++) {
    if (patientArray[i].age > 60) {
        console.log("Found patient with age > 60:", patientArray[i].name);
        console.log("Stopping loop at this point...");
        break;
    }
    console.log("Checking:", patientArray[i].name, "(Age:", patientArray[i].age + ")");
}

console.log("\n=== USING CONTINUE (Skip specific disease) ===\n");

for (let i = 0; i < patientArray.length; i++) {
    if (patientArray[i].disease === "Diabetes") {
        console.log("Skipping:", patientArray[i].name, "-", patientArray[i].disease);
        continue;
    }
    console.log("Processing:", patientArray[i].name, "-", patientArray[i].disease);
}
```

Output –

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
=== USING CONTINUE (Skip admitted patients) ===

Non-admitted Patient: Robert Smith
Non-admitted Patient: Sarah Wilson

=== USING BREAK (Stop at age > 60) ===

Checking: Alice Johnson (Age: 35)
Checking: Robert Smith (Age: 58)
Found patient with age > 60: Maria Garcia
Stopping loop at this point...

=== USING CONTINUE (Skip specific disease) ===

Processing: Alice Johnson - Migraine
Skipping: Robert Smith - Diabetes
Processing: Maria Garcia - Arthritis
Processing: David Brown - Fracture
Processing: Sarah Wilson - Hypertension

[Done] exited with code=0 in 0.222 seconds
```

Brief Description:

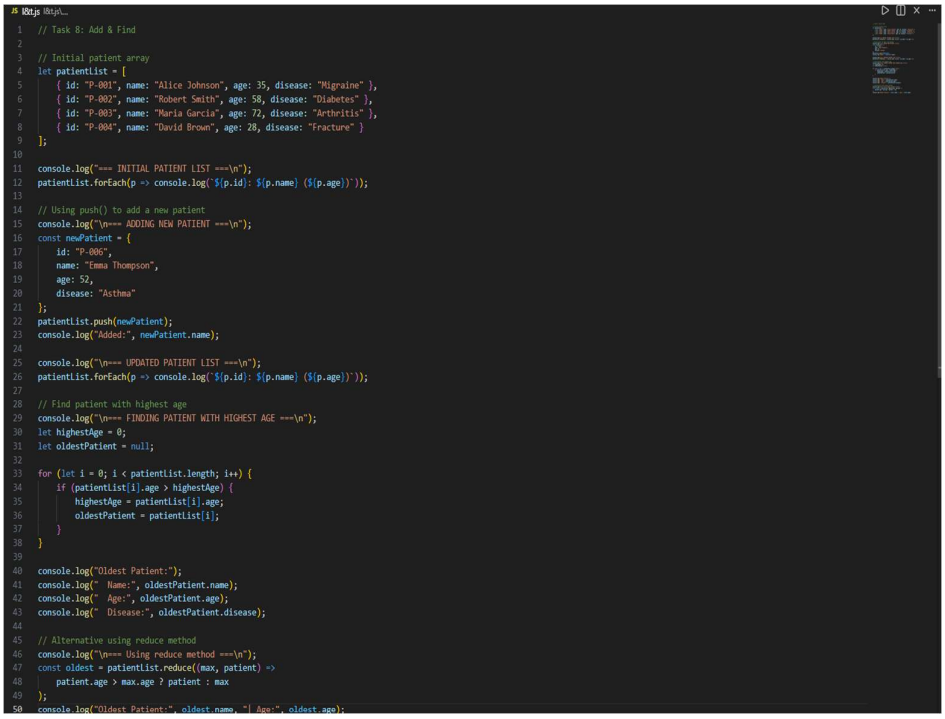
This task demonstrates loop control flow using break and continue statements. The patient array is iterated, and specific conditions are applied: continue is used to skip admitted patients or patients with specific diseases, while break is used to stop the loop when a patient above a certain age is found. This showcases understanding of conditional loop control and selective data processing.

Task 8: Hospital Patient System — Add & Find 2 Marks

→ Use push() to add a new patient to the array.

→ Write a loop to find the patient with the highest `age`.

Answer –



```
// Task 8: Add & Find

// Initial patient array
let patientList = [
    { id: "P-001", name: "Alice Johnson", age: 35, disease: "Migraine" },
    { id: "P-002", name: "Robert Smith", age: 58, disease: "Diabetes" },
    { id: "P-003", name: "Maria Garcia", age: 72, disease: "Arthritis" },
    { id: "P-004", name: "David Brown", age: 28, disease: "Fracture" }
];

console.log("=== INITIAL PATIENT LIST ===\n");
patientList.forEach(p => console.log(`${p.id}: ${p.name} (${p.age})`));

// Using push() to add a new patient
console.log("\n=== ADDING NEW PATIENT ===\n");
const newPatient = {
    id: "P-006",
    name: "Emma Thompson",
    age: 52,
    disease: "Asthma"
};
patientList.push(newPatient);
console.log("Added:", newPatient.name);

console.log("\n=== UPDATED PATIENT LIST ===\n");
patientList.forEach(p => console.log(`${p.id}: ${p.name} (${p.age})`));

// Find patient with highest age
console.log("\n=== FINDING PATIENT WITH HIGHEST AGE ===\n");
let highestAge = 0;
let oldestPatient = null;

for (let i = 0; i < patientList.length; i++) {
    if (patientList[i].age > highestAge) {
        highestAge = patientList[i].age;
        oldestPatient = patientList[i];
    }
}

console.log("Oldest Patient:");
console.log("  Name:", oldestPatient.name);
console.log("  Age:", oldestPatient.age);
console.log("  Disease:", oldestPatient.disease);

// Alternative using reduce method
console.log("\n=== Using reduce method ===\n");
const oldest = patientList.reduce((max, patient) =>
    patient.age > max.age ? patient : max
);
console.log("Oldest Patient:", oldest.name, "| Age:", oldest.age);
```

Output –

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
=== INITIAL PATIENT LIST ===

P-001: Alice Johnson (35)
P-002: Robert Smith (58)
P-003: Maria Garcia (72)
P-004: David Brown (28)

=== ADDING NEW PATIENT ===

Added: Emma Thompson

=== UPDATED PATIENT LIST ===

P-001: Alice Johnson (35)
P-002: Robert Smith (58)
P-003: Maria Garcia (72)
P-004: David Brown (28)
P-006: Emma Thompson (52)

=== FINDING PATIENT WITH HIGHEST AGE ===

Oldest Patient:
  Name: Maria Garcia
  Age: 72
  Disease: Arthritis

=== Using reduce method ===

Oldest Patient: Maria Garcia | Age: 72

[Done] exited with code=0 in 0.117 seconds
```

Brief Description:

This task combines array manipulation with data analysis. The push() method is used to add a new patient object to the existing array. Then, a loop is written to find the patient with the highest age by comparing each patient's age. The reduce() method is also demonstrated as an alternative approach. This showcases array operations, object manipulation, and algorithmic thinking for finding maximum values.

Task 9: Hospital Patient System — Remove & Sort 2 Marks

→ Use pop() or shift() to remove an entry from the array.

→ Use the array's sort() method to sort patients by `age`.

Answer –

l&t.js l&t.js\...

```js
// Task 9: Remove & Sort

// Patient array
let patientData = [
    { id: "P-001", name: "Alice Johnson", age: 35, disease: "Migraine" },
    { id: "P-002", name: "Robert Smith", age: 58, disease: "Diabetes" },
    { id: "P-003", name: "Maria Garcia", age: 72, disease: "Arthritis" },
    { id: "P-004", name: "David Brown", age: 28, disease: "Fracture" },
    { id: "P-005", name: "Sarah Wilson", age: 45, disease: "Hypertension" }
];

console.log("=== INITIAL PATIENT LIST ===\n");
patientData.forEach(p => console.log(`${p.id}: ${p.name} (${p.age})`));

// Using pop() to remove last entry
console.log("\n=== USING pop() TO REMOVE LAST ENTRY ===\n");
const removedPatient = patientData.pop();
console.log("Removed:", removedPatient.name);
console.log("Remaining count:", patientData.length);

// Using shift() to remove first entry
console.log("\n=== USING shift() TO REMOVE FIRST ENTRY ===\n");
const removedFirst = patientData.shift();
console.log("Removed first:", removedFirst.name);
console.log("Remaining count:", patientData.length);

console.log("\n=== AFTER REMOVAL ===\n");
patientData.forEach(p => console.log(`${p.id}: ${p.name} (${p.age})`));

// Sort patients by age
console.log("\n=== SORTING PATIENTS BY AGE (Ascending) ===\n");
const sortedByAge = [...patientData].sort((a, b) => a.age - b.age);
sortedByAge.forEach(p => console.log(`${p.id}: ${p.name} (${p.age})`));

console.log("\n=== SORTING PATIENTS BY AGE (Descending) ===\n");
const sortedDescending = [...patientData].sort((a, b) => b.age - a.age);
sortedDescending.forEach(p => console.log(`${p.id}: ${p.name} (${p.age})`));

console.log("\n=== SORTING PATIENTS BY NAME (Alphabetical) ===\n");
const sortedByName = [...patientData].sort((a, b) => a.name.localeCompare(b.name));
sortedByName.forEach(p => console.log(`${p.id}: ${p.name} (${p.age})`));
```

Output-

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
=== INITIAL PATIENT LIST ===

P-001: Alice Johnson (35)
P-002: Robert Smith (58)
P-003: Maria Garcia (72)
P-004: David Brown (28)
P-005: Sarah Wilson (45)

=== USING pop() TO REMOVE LAST ENTRY ===

Removed: Sarah Wilson
Remaining count: 4

=== USING shift() TO REMOVE FIRST ENTRY ===

Removed first: Alice Johnson
Remaining count: 3

=== AFTER REMOVAL ===

P-002: Robert Smith (58)
P-003: Maria Garcia (72)
P-004: David Brown (28)

=== SORTING PATIENTS BY AGE (Ascending) ===

P-004: David Brown (28)
P-002: Robert Smith (58)
P-003: Maria Garcia (72)

=== SORTING PATIENTS BY AGE (Descending) ===

P-003: Maria Garcia (72)
P-002: Robert Smith (58)
P-004: David Brown (28)

=== SORTING PATIENTS BY NAME (Alphabetical) ===

P-004: David Brown (28)
P-003: Maria Garcia (72)
P-002: Robert Smith (58)

[Done] exited with code=0 in 0.15 seconds
```

Brief Description:

This task focuses on array modification and sorting operations. The pop() method removes the last element, while shift() removes the first element from the patient array. The sort() method is then used to sort patients by age in both ascending and descending order. Sorting by name is also demonstrated. This showcases understanding of array manipulation methods, sorting algorithms, and custom comparator functions.

Task 10: Hospital Patient System — Create and Print an Object 2 Marks

→ Create a `patient` object with at least 5 properties and print each property individually**.**

Answer –

JS l&t.js l&t.js\...

```
// Task 10: Create and Print an Object

// Creating a patient object with 5+ properties
const patient = {
    id: "P-2026-001",
    name: "Jennifer Martinez",
    age: 42,
    gender: "Female",
    disease: "Pneumonia",
    admitted: true,
    admissionDate: "2026-07-20",
    contactNumber: "+91-9876543210",
    insuranceProvider: "Medicare",
    roomNumber: 305
};

console.log("=== PATIENT DETAILS ===\n");

// Printing each property individually
console.log("Patient ID:", patient.id);
console.log("Patient Name:", patient.name);
console.log("Age:", patient.age);
console.log("Gender:", patient.gender);
console.log("Disease:", patient.disease);
console.log("Admitted Status:", patient.admitted ? "Yes" : "No");
console.log("Admission Date:", patient.admissionDate);
console.log("Contact Number:", patient.contactNumber);
console.log("Insurance Provider:", patient.insuranceProvider);
console.log("Room Number:", patient.roomNumber);

// Printing all properties using a loop
console.log("\n=== ALL PROPERTIES (Using for...in loop) ===\n");
for (let key in patient) {
    console.log(`${key}: ${patient[key]}`);
}

// Printing as a formatted JSON
console.log("\n=== AS JSON FORMAT ===\n");
console.log(JSON.stringify(patient, null, 2));
```

Output-

```
[Running] node "d:\SEM5\CW L&T lab\l&t.js"
=== PATIENT DETAILS ===

Patient ID: P-2026-001
Patient Name: Jennifer Martinez
Age: 42
Gender: Female
Disease: Pneumonia
Admitted Status: Yes
Admission Date: 2026-07-20
Contact Number: +91-9876543210
Insurance Provider: Medicare
Room Number: 305

=== ALL PROPERTIES (Using for...in loop) ===

id: P-2026-001
name: Jennifer Martinez
age: 42
gender: Female
disease: Pneumonia
admitted: true
admissionDate: 2026-07-20
contactNumber: +91-9876543210
insuranceProvider: Medicare
roomNumber: 305

=== AS JSON FORMAT ===

{
  "id": "P-2026-001",
  "name": "Jennifer Martinez",
  "age": 42,
  "gender": "Female",
  "disease": "Pneumonia",
  "admitted": true,
  "admissionDate": "2026-07-20",
  "contactNumber": "+91-9876543210",
  "insuranceProvider": "Medicare",
  "roomNumber": 305
}
```

Brief Description:

This task involves creating a comprehensive patient object with multiple properties (at least 5, but more are added for completeness). Properties include patient ID, name, age, gender, disease, admission status, admission date, contact number, insurance provider, and room number. Each property is printed individually using dot notation. Additionally, for...in loop and JSON formatting are demonstrated for alternative printing methods. This showcases object creation, property access, and data organization in JavaScript.

NAME: AMOGH. V. HOSAMANI
Reg No: 2462544
CLASS: 5 BTCS10T

SET B

1. Write a program to check whether a given year is leap year using if-else

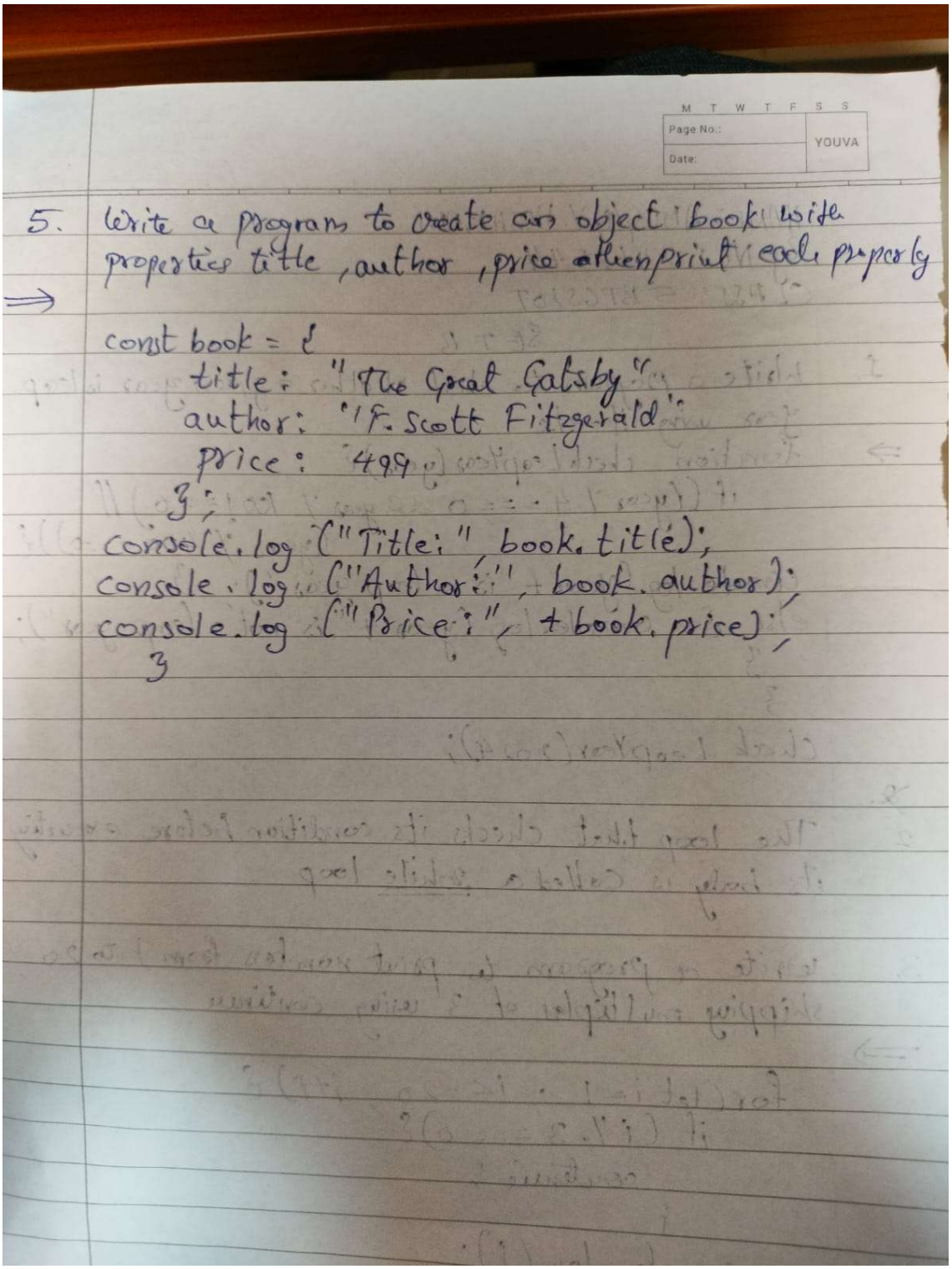
⇒
```
function checkLeapYear(years){
    if((year % 4 === 0 && year % 100 !== 0) ||
                (years % 400 === 0)){
console.log(year + "is a leap year");
else { console.log(year + " is not a leap year");
  }
}
checkLeapYear(2024);
```

2. The loop that checks its condition before executing its body is called a while loop

3. Write a program to print numbers from 1 to 20, skipping multiples of 3 using continue

⇒
```
for(let i=1 ; i<=20; i++){
    if (i % 3 === 0){
        continue;
    }
    console.log(i);
}
```

4. The method used to add a new element to the beginning of an array is unshift()



5. Write a program to create an object 'book' with properties title, author, price and print each property

⇒

```
const book = {
    title: "The Great Gatsby",
    author: "F. Scott Fitzgerald",
    price: 499
};
console.log("Title:", book.title);
console.log("Author:", book.author);
console.log("Price:", +book.price);
```

GITHUB LINK - https://github.com/amoghvh/L-t